Technical Design Document

Vantasona : Sonata Of Silence

1. **Project Overview :**

- Prisma is a prototype game project developed in Unity.
-The goal of the project is to experiment with gameplay systems, player movement and camera behavior and a world color restoration mechanic.

-In the current stage, the project focuses on building the core prototype systems before implementing the full gameplay loop.

2. Current Implemented Systems :

2.1 Player Movement System :

The player movement system is implemented using Unity's Rigidbody physics and the new Input System.

Current functionality:

- Movement based on player input
- Camera-relative movement direction
- Jump mechanic
- Ground detection
- Player rotation based on movement direction
- Movement acceleration and air control

The system is implemented in the script:

-- **PlayerMovementRB_AAA:**

Responsibilities of the script:

- Read player input
- Apply movement forces to the Rigidbody
- Handle jump logic

- Read player input
- Apply movement forces to the Rigidbody
- Handle jump logic

2.2 World Color Prototype System :

The project experiments with a gameplay concept where the world is originally colored but appears grey to the player at the start of the game.

Current idea:

- The world contains colors and textures
- At the start of the game, the world appears grey
- Only the player keeps their original colors
- When the player defeats enemies, color is gradually restored around the player

This system is currently in the prototype phase and not fully implemented yet.

Scripts related to this concept include:

- WorldColorManager
- shader and material experiments

shader and material experiments

3. Scene Structure

Main objects currently used in the scene:

- Main Camera
- Directional Light
- Player
- Character Visual Model
- FreeLook Camera
- WorldColorManager
- MapLevelOne
- Ground and environment objects

These objects are used to test gameplay systems during development.

4. Current Development State

Working Systems

- Player movement
- Jump system
- Camera integration
- Character model integration
- Basic animation playback
- Test environment

Systems Under Development

- World greyscale system
- Color restoration mechanic around the player
- Enemy interaction with the color system

Not Implemented Yet

- Final gameplay loop
- Enemy combat system
- Music note collection mechanic
- Final environment assets
- Final shader system

5. Tools and Technologies

The project currently uses:

- Unity 6
- Universal Render Pipeline (URP)
- Unity Input System
- Rigidbody Physics
- Prototype shaders and materials

6-The next development goals include:

1. Implement the world greyscale effect properly.
2. Create the color restoration radius around the player.
3. Increase the restoration radius after defeating enemies.
4. Restore the full world color after defeating the final boss.
5. Refactor scripts and organize project architecture.

7- Platforms and Hardware Specifications

The project is currently developed as a **PC prototype** using the Unity engine.

a- ***Target platform :*** Windows PC

b- Development hardware used for testing:

CPU : i7-4790

GPU : GTX-1650

RAM : 16

STORAGE : SSD 128GB

Since the project is currently a **prototype**, hardware requirements are kept moderate. Optimization for other platforms is not considered at this stage.

8- Development Environment

The project is developed using the following tools and technologies.

Game Engine

- **Unity 6**

Programming Language

- **C#**

Rendering Pipeline

- **Universal Render Pipeline (URP)**

Version Control

- **Git**

IDE / Code Editor

- **Visual Studio / VS Code**

3D Assets and Models

- Prototype assets and placeholder models

Game Design Tools

- Unity Editor
- Unity Scene tools

9- Technical Risks

Several technical risks were identified during development.

Rhythm System Synchronization

The game concept involves rhythm-based mechanics.
If the synchronization between **music timing and player input** is inaccurate, gameplay may feel unfair.

Possible solution:

- Using timing windows for input detection
- Central rhythm manager controlling note timing

World Color Restoration System

The color restoration mechanic requires shader and rendering manipulation.

Potential issues:

- Shader performance
- Material compatibility

Possible solution:

- Prototype with simple shaders
- Gradually extend visual complexity.

10- Production Pipeline

The current development pipeline follows a simple structure suitable for a prototype project.

Development workflow:

1. Game design concept
2. System prototyping in Unity
3. Script implementation
4. Scene integration
5. Testing and iteration

Assets workflow:

1. Create or import temporary assets
2. Test assets inside the Unity scene
3. Replace with improved versions later

Version control is used to synchronize work between team members.

11- Software Architecture

The architecture follows a modular system structure to avoid large monolithic scripts.

Main systems include:

Player System

Handles player behavior:

Components:

- Player movement
- Jump system
- Input reading
- Rotation control

Main script:

- PlayerMovementRB_AAA

Camera System

Responsible for camera control and player perspective.

Uses:

- Cinemachine FreeLook camera
- Camera following the player
- Rotation based on mouse input

World System

Handles environmental mechanics.

Components:

- Map objects
- Ground and level structures
- Color restoration system

Managed by:

- WorldColorManager

Enemy System

Handles enemies and combat interactions.

Responsibilities:

- Enemy spawning
- Enemy defeat detection
- Trigger gameplay events.

UI System (future implementation)

Planned UI elements include:

- Rhythm prompts
- Combat feedback
- Game menus.

12- Performance Considerations

Since the project is a prototype, performance optimization is limited but still monitored.

Target performance:

- 60 FPS on development hardware

Performance tools:

- Unity Profiler
- Unity Stats window

Potential performance issues:

- shader effects
- physics calculations
- enemy spawning.

Optimization strategies:

- limit unnecessary Update calls
- reduce physics checks frequency

13- Gameplay Mechanisms Technical Implementation

The core gameplay concept is based on restoring sound and color to the world.

Color Restoration Mechanic

At the start of the game:

- The world appears grey
- The player remains colored

Gameplay logic:

1. Player explores the world
2. Player defeats enemies
3. Defeating enemies triggers color restoration
4. Color gradually returns to nearby areas

This mechanic is managed by:

- WorldColorManager
- shader / material experiments

Player Movement Mechanic

The movement system uses **Unity Rigidbody physics**.

Movement process:

1. Player input is read using Unity Input System
2. Input direction is converted relative to camera direction
3. Movement force is applied to Rigidbody
4. Player rotates toward movement direction
5. Jump is triggered when grounded.

Future Rhythm Combat Mechanic

The combat system will involve rhythm gameplay elements.

Planned logic:

1. Enemy encounter triggers rhythm combat
2. Notes appear on the screen
3. Player must press inputs in correct timing
4. Successful inputs damage enemies
5. Defeating enemies restores sound layers.

14- Team Organization

The project is developed by a small team.

Each team member is responsible for specific systems.

Example responsibilities:

- Gameplay systems
- Camera and movement
- Environment mechanics
- UI systems

Version control is handled using **Git**.

Workflow:

1. Each developer works on their own branch
2. Changes are tested
3. Changes are merged into the main project.